Unit 2 Progress Check: FRQ Part B

This question involves processing strings of text.

Consider the following variables.

- colWidth is an int variable with a positive value.
- text is a String variable with a non-null value.
- otherText is a String variable with a non-null value.

Unit 2 Progress Check: FRQ Part B

1. A code segment, which you will write, should print output according to the following rules.

- If the length of text is greater than colWidth, then the value of text will be printed.
- Otherwise, the printed output will consist of colWidth characters contained between two "|" characters. The text between the "|" characters will consist of a sequence of periods ("."), followed by the value of text, followed by another sequence of periods.

  The printed output will have an equal number of periods on each side of text, if possible. If this is not possible, then there will be one fewer period on the left side of text than on the right side of text.

The intended behavior is illustrated in the following table.

Value of colWidth	Value of text	Printed Output
5	**"ab"**	|.ab..|
5	**"abc"**	|.abc.|
10	**"ab"**	|....ab....|
10	**"abc"**	|...abc....|
10	**"abcdefghij"**	|abcdefghij|
10	**"abcdefghijklmnopqrstuv"**	abcdefghijklmnopqrstuv

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Prints the original text if the text length is longer than the width of the column

Unit 2 Progress Check: FRQ Part B

Canonical Solution:

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method
- use an incorrect comparison

Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies the number of periods needed before and after the text

Canonical Solution:

Unit 2 Progress Check: FRQ Part B

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly determine the length of the text

Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Prints string in appropriate format with identified number of periods before and after the text (*algorithm*)

Canonical Solution:

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
```

Unit 2 Progress Check: FRQ Part B

```
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

0	✓ 1

The response earns the point.

Responses can still earn the point if they

- print an incorrect number of periods before and after the text

Unit 2 Progress Check: FRQ Part B

2. A code segment, which you will write, should print the length of the longest prefix of the text that is a substring of otherText. A prefix is a substring that occurs at the beginning of another string.

The intended behavior is illustrated in the following table.

Value of text	Value of otherText	Printed Output	Explanation
"apple"	**"pear"**	1	The prefix "a" is a substring of "pear", but "ap" is not.
"apple"	**"pineapple"**	5	The prefixes "a", "ap", "app", "appl", and "apple" are substrings of "pineapple". The length of the longest substring is 5.
"apple"	**"happy"**	3	The prefixes "a", "ap", and "app" are substrings of "happy". The length of the longest substring is 3.
"apple"	**"leek"**	0	No prefix of "apple" is a substring of "leek".

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loop bounds support accessing all necessary parts of `text`

Canonical Solution:

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
```

Unit 2 Progress Check: FRQ Part B

```
   {
      longest = len;
   }
}
System.out.println(longest);
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- use either a condition or an update that would cause an infinite loop or skipped elements in some cases

Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Uses an appropriate `String` method to determine whether `text` is a substring of `otherText`

Canonical Solution:

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
   {
      longest = len;
   }
}
System.out.println(longest);
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

Unit 2 Progress Check: FRQ Part B

- make any incorrect call to a `String` method

Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines the length of the longest prefix (*algorithm*)

Canonical Solution:

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
   {
      longest = len;
   }
}
System.out.println(longest);
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to print the result
- loop an incorrect number of times
- make an incorrect call to a `String` method

Responses will not earn the point if they

- fail to update a longest length variable

Unit 2 Progress Check: FRQ Part B

This question involves writing two code segments that use information about downloaded songs.

A record company uses the following variables to maintain information about each song in the company's catalog.

- `downloads` is an `int` variable that represents the number of times a song was downloaded. Assume the value of this variable is positive.
- `songInfo` is a `String` variable that contains information about the song. The string consists of the artist name, followed by a hyphen ("-"), followed by the album title, followed by another hyphen, followed by the song title. Assume that no other hyphens appear in the string.

3. A code segment, which you will write, should determine whether the ones digit of the number stored in `downloads` appears exactly one other time in the number. If so, the code segment should print `"exactly once"`; otherwise, it should print `"not exactly once"`.

For example, if `downloads` has a value of `1734567`, then the segment should print `"exactly once"` because the ones digit, which is 7, appears one other time.

As another example, if `downloads` has a value of `325322`, then the segment should print `"not exactly once"` because the ones digit, which is 2, appears two other times.

Write a code segment to print the intended output. Your implementation must conform to the examples.

Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies at least one digit in `downloads`

Canonical Solution:

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
```

Unit 2 Progress Check: FRQ Part B

```
}

if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- do not store the value of the ones digit in a variable

Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all necessary digits in `downloads`

Canonical Solution:

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
}
```

Unit 2 Progress Check: FRQ Part B

```
if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to initialize counter

Responses will not earn the point if they

- fail to update counter

Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines if `downloads` has exactly one occurrence of the ones digit

Canonical Solution:

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
}
```

Unit 2 Progress Check: FRQ Part B

```
if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to print the message

Unit 2 Progress Check: FRQ Part B

4. A code segment, which you will write, should process the string songInfo and print the song title and artist name separated by the string (" by ").

For example, if songInfo is "Carrie Ferriero-Greatest Hits-Tell Me Why", then the code segment should print "Tell Me Why by Carrie Ferriero".

As another example, if songInfo is "Aniita-Mixtape 2-How Much More", then the code segment should print "How Much More by Aniita".

Write a code segment to print the intended output. Your implementation must conform to the examples.

Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies the artist name from the song information string

Canonical Solution:

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- incorrectly identify the location of the first hyphen ("-")

Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

Unit 2 Progress Check: FRQ Part B

- Identifies the song title from the artist information string

Canonical Solution:

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly identify the location of the second hyphen ("-")

Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Correctly prints the identified artist and song title

Canonical Solution:

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

Unit 2 Progress Check: FRQ Part B

- incorrectly identify the artist name or song title

Responses will not earn the point if they

- print the artist and song title in an incorrect format